

Java & Spring Boot Developers ke liye Beginner se PRO tak

Hindi mein samjhaaya gaya — har topic ke saath real-life examples, commands, aur key points. Padhne aur yaad karne ke liye ready-to-use notes.

Table of Contents (Vishay Suchi)

1. Problem: "It Works on My Machine" — Akhir problem kya hai?
2. Environment Mismatch Problem (Mlaabh ka kaaran)
3. Docker to the Rescue — Solution kya hai?
4. Docker Image kya hai? (Image ki poorn jaankari)
5. Docker Container kya hai? (Isolated environment)
6. Image vs Container — Difference samjho
7. Docker Install karna aur Docker Architecture
8. Pehla Docker App — hello-world
9. Docker Hub — Images ka Marketplace
0. Docker Tags Explained (version system)
1. "latest" Tag ka asli matlab
2. docker ps — Running containers dekhna
3. Containers apne aap kyun stop hote hain?
4. Interactive Mode (-it flag) — Container ke andar jaana
5. OS, Kernel & User Space — Basics
6. Virtual Machines vs Docker (Hypervisor)
7. Docker Shared Kernel kaam kaise karta hai
8. Ubuntu Container chalana aur explore karna
9. Spring Boot App ko Dockerize karna — Dockerfile
0. Docker Image Build karna
1. Port Mapping Explained (-p flag)
2. Docker Hub par Login aur Image Push karna
3. Image Naming Convention
4. Detached Mode (-d) — Background mein chalana
5. Docker + Microservices = Perfect Match
6. Docker Images are Immutable (nahi badalti)
7. Essential Docker Commands (stop, rm, prune)
8. Quick Revision Summary (Ek nazari mein)

1. Problem: "It Works on My Machine" — Aakhir problem kya hai?

Maan lo aap ek Java Spring Boot developer ho. Aapne ek project banaya jiska naam hai **Employee Service**. Aapne is project ko banane ke liye **Java 26** aur **Spring Boot 4** ka use kiya. Aapne apne laptop par application chalayi, ek REST endpoint banaya (`/employee`), aur jab aapne browser mein `localhost:8080/employee` hit kiya toh sahi se output mil gaya — Employee id=1, name=Ayush.

Ab aapne is application ka **JAR file** banaya (Maven ke through `mvn install` ya `./mvnw install` chala kar) aur us JAR ko apni team ke dusre members ko de diya — dev team, testing team, sab ko. Aapka sochna tha ki "Java toh Write Once Run Anywhere hai, toh ye JAR sab ke laptop par chal jayegi."

Lekin sachai ye hai ki JAR dusre laptop par NAHI chalegi. Kyunki Java ka tagline "Write Once Run Anywhere" sirf tab kaam karta hai jab dono machine par same version ka Java installed ho.

✂ Real-Life Example (Hindi):

Sochiye aapne ek **khaas recipe ki video** banayi (jaise masala dosa banane ki). Us video ko aapne apne dost ko WhatsApp par bhej diya. Lekin aapke dost ke paas **wahi pressure cooker nahi hai** jo aapke paas hai, **wahi gas stove nahi hai**, aur **wahi masale ke brand nahi hain**. Toh dost ne video toh dekh li, lekin dosa waisa nahi bana. Kyunki **environment alag tha**. Bilkul waise hi — aapne JAR banayi Java 26 se, lekin dost ke laptop par Java 25 hai, toh JAR chalegi hi nahi.

⚠ Error jo aayega:

"Unsupported class file version" — iska matlab ye JAR higher Java version se compile hui hai aur aapke paas ki Java purani hai. Ye **application ki problem nahi, environment ki problem hai**.

🔔 Yaad rakho:

Famous developer meme hai — "Who are we? Software engineers. Where does our software work? On our laptop. Where should it work? On production. Does it work there? No." Ye bilkul sach hai. Software aapke laptop par chalta hai, production par nahi, kyunki production machine ka environment alag hota hai.

2. Environment Mismatch Problem (Milan ka kaaran)

Video mein instructor ne do alag laptops liye — ek Mac Mini (jisme Java 26 installed tha) aur ek MacBook Pro (jisme Java 25 installed tha). Jab Mac Mini se banayi gayi JAR ko MacBook Pro par `java -jar employee-service.jar` se chalaya, toh error aaya. MacBook Pro ne bola: "ye class file Java 26 se bani hai, main sirf Java 25 tak pehchanta hoon."

Ye sirf Java version ki problem nahi hai. Aapke application ko chalane ke liye aur bhi cheezein chahiye ho sakti hain — **MySQL 8** (database ke liye), **Kafka 4** (messaging ke liye), **MongoDB 8** (NoSQL ke liye). Toh har team member ke laptop par ye sab same version ke saath installed hone chahiye, warna application nahi chalegi.

✂ Real-Life Example (Hindi):

Sochiye ek **bada shaadi ka ghar** (banquet hall) hai jahan khana banana hai. Main cook (developer) ne recipe banayi, lekin us recipe ko chalane ke liye chahiye — **specific microwave** (Java version), **specific refrigerator** (MySQL), **specific mixer grinder** (Kafka), aur **specific oven temperature** (MongoDB). Agar dusre kitchen mein ye saare appliances **same brand aur same model ke nahi hain**, toh recipe ka result alag aayega ya bilkul nahi aayega. Har helper ko (team member ko) ye saare appliances exactly waise hi lene padenge jo main cook ke paas the — aur ye bahut time-consuming aur painful hai.

🔑 Key Points:

- Pehle companies ek **"installation guide" document** deti thi naye employees ko — Java 26 install karo, MySQL 8 install karo, Kafka 4 install karo, phir JAR chalao.
- Naya employee sirf application ko start karne mein **10-15-20 din** laga deta tha — aur company usko salary de rahi thi sirf setup ke liye!
- Dev team, test team, deployment team — sab ko ye **"traumatic experience"** se guzarna padta tha.
- Software sirf localhost par nahi, **har jagah** chalna chahiye — yahi problem fix karni hai.

3. Docker to the Rescue — Solution kya hai?

Problem ka solution ye hai ki hum sirf JAR file nahi denge, balki **pura environment** denge. Matlab JAR ke saath — Java 26, MySQL 8, Kafka 4, MongoDB 8 — sab kuch ek saath package karke denge. Phir dusre laptop par ye sab kuch ek saath aa jayega aur application turant chal jayegi.

Ye kaam karne ke liye ek software hai jiska naam hai **Docker**. Docker humein madad karta hai sab kuch ek saath package karne mein. Jab hum sab kuch package kar lete hain, toh us packaged bundle ko **Docker Image** kehte hain.

✂ Real-Life Example (Hindi):

Sochiye aapko apne gaon ke **pyaar ke saath ek "thali" (plate)** bhejni hai. Pehle aap alag-alag dabbe mein roti, sabzi, daal, chawal, achaar bhejte the — aur pata nahi chalta ki kya-kya chahiye. Ab aap ek **"complete tiffin box"** bana lete hain jisme sab kuch properly arranged hai — roti ek compartment mein, sabzi dusre mein, daal teesre mein, aur chawal chauthre mein. Saath mein spoon bhi hai. Ab jis ko bhi ye tiffin milega, woh turant khana kha sakta hai, kuch aur install/mangwane ki zaroorat nahi. **Docker Image bilkul waisa hi "complete tiffin" hai** — JAR + Java + saare dependencies ek box mein.

🔔 Important line:

"Ab se JAR file mango mat — Docker Image mango." (Don't ask for JAR anymore — ask for Docker Image.)

4. Docker Image kya hai? (Image ki poorn jaankari)

Docker Image ek **blueprint** hai, ek **package** hai. Isme aapki application (JAR) hai, us JAR ko chalane ke liye zaroori Java (JDK) hai, OS environment hai (jaise Linux Alpine ya Ubuntu), aur saare dependencies hain. Sab kuch ek saath packaged hai — bilkul ek zip file ki tarah soch sakte ho.

Lekin image akele kuch nahi karti. Bilkul waise hi jaise aapke laptop par JAR file rakhne se application nahi chalti — JAR ko `java -jar` se chalana padta hai. Waise hi Image ko bhi **run** karna padta hai. Jab hum image ko run karte hain, tab ek **container** banta hai, aur us container ke andar humein desired environment milta hai.

✂ Real-Life Example (Hindi):

Image = Building ka Naksha (Blueprint). Jab architect kisi ghar ka naksha banata hai (blueprint), toh woh naksha akele kuch nahi karta — woh sirf ek plan hai. Jab builder us nakshe ke according **asli ghar banata hai** (construction karta hai), tab ghar mein rehaya ja sakta hai. **Naksha = Docker Image, Asli Ghar = Docker Container.** Ek nakshe se aap chaaron ghar bana sakte ho — ek image se aap chaaron container bana sakte ho.

🔔 Image ke baare mein yaad rakho:

- Image ek **blueprint/package** hai — akele kuch nahi karti.
- Image mein hai: Application (JAR) + JDK + OS + saare dependencies.
- Image ko **run** karna padta hai tabhi container banta hai.
- Ek image se aap **chaaron container** bana sakte ho.
- Different companies apni images publish karti hain (Java, Node, MySQL, Python, etc.).

5. Docker Container kya hai? (Isolated environment)

Maan lo aap ek company mein join kiye, aur company ke paas 4-5 alag-alag applications hain — ek Java (Java 26 chahiye), ek JavaScript (Node 20 chahiye), ek Python (Python chahiye), aur ek legacy Java app (purana Java 1.8 chahiye). Sab applications ko aapne laptop par chalana hai.

Agar aap sab kuch **same laptop par direct install karte ho**, toh problem aayegi — ek application ko MySQL 6 chahiye, dusre ko MySQL 7, teesre ko Linux environment, chauthre ko Ubuntu. Sab kuch ek hi machine par daal do toh **bahut bada mess ban jayega**. Alag-alag versions ek dusre se conflict karenge.

Iska solution hai — **har application ko apne alag "container" mein daal do**. Har container ek isolated environment hai. Ek

container dusre container ko nahi dekh sakta. Isi liye Docker ka logo bhi **dibbe (boxes)** jaisa hai — har dibba alag environment.

🚢 **Real-Life Example (Hindi) — Ship aur Containers:**

Sochiye ek **bada cargo ship** hai jisme banana, orange, laptop, mobile, saaman rakha hai. Agar sab ek saath bina organization ke rakha jaye, toh bahut messy hai. Lekin agar har cheez ko alag **"shipping container"** (dibbe) mein rakha jaye — laptop ke liye alag box, mobile ke liye alag box, aur apple/banana ke liye alag box jisme **refrigerator** (cooling) ho — toh sab organized hai. Apple wale box mein cooling hai, laptop wale box mein nahi — kyunki usko nahi chahiye. **Docker container bilkul waisa hi hai** — har application ko uski zaroorat ke according isolated environment milta hai.

🍳 **Real-Life Example (Hindi) — Kitchen:**

Ek bada kitchen sochiye jahan **alag-alag sections** hain — ek section mein chai ban rahi hai (gas stove + kettle), dusre section mein roti ban rahi hai (tava), teesre section mein chicken fry ho raha hai (kadahi + oil). Har section ka apna equipment hai, aur ek section ka saaman dusre section mein nahi milta. Agar sab kuch ek hi table par daal dete toh confusion hota. **Har Docker container ek "section" ki tarah hai** — apni zaroorat ke tools ke saath, dusre se isolated.

🔑 **Container ke baare mein:**

- Container ek **isolated environment** hai jisme sirf us application ki zaroorat ke software hote hain.
- Ek container dusre container ko **nahi dekh sakta** (isolated).
- Aap ek hi laptop par **chaaron alag-alag container** chala sakte ho.
- Docker logo (boxes) isi concept ko darshata hai.
- Container image se **banaya** jata hai (image ko run karke).

Topic 6

6. Image vs Container — Difference samjho

Ye sabse important concept hai jo bahut log confuse karte hain. Aaiye clear karte hain.

Docker Image	Docker Container
Ye ek blueprint/package hai (read-only).	Ye image ko run karke bana running environment.
Image akele kuch nahi karti .	Container actively chalta hai (application run hoti hai).
Image ek hi baar bani, phir baar-baar use ho sakti hai.	Ek image se chaaron container banaye ja sakte hain.
Static hai — disk par stored.	Dynamic hai — memory mein running.
Jaise: Roti ka aata (raw material).	Jaise: Aate se bani paki hui roti .

🚢 **Real-Life Example (Hindi):**

Image = DVD par stored **movie** (jab tak chalayoge nahi, dekh nahi sakte). **Container** = Us movie ko player mein **chalana** (ab movie chal rahi hai, dekh sakte ho). Ek DVD se aap chaaron baar movie chala sakte ho — har baar ek naya "running instance" banta hai (container).

🔄 **Simple flow yaad rakho:**

Image → (docker run) → **Container** (running environment)

Topic 7

7. Docker Install karna aur Docker Architecture

Docker ko install karne ke liye browser mein **Docker for Desktop** search karo. Apne OS ke according download karo — Mac (Apple Silicon / Intel), Windows, ya Linux. Install karne ke baad Docker Desktop app chalu karo. Terminal mein **docker --version** likho — agar version aaya toh Docker successfully install ho gaya.

Docker Architecture (3 main parts):

- **Docker Host:** Ye aapka laptop/machine hai jis par Docker install hai.

- **Docker Server (Daemon):** Ye background mein chalne wala engine hai jo actually containers banata hai aur manage karta hai.
- **Docker CLI:** Ye command-line tool hai jis se aap Docker server ko commands dete ho. Aap terminal mein `docker run`, `docker ps` jaisi commands likhte ho, ye commands server (daemon) tak pahunchti hain.

Server ke saath interact karne ke do tareeke hain — **Docker CLI** (terminal se) aur **Docker Desktop GUI** (visual interface se).

🔗 **Real-Life Example (Hindi):**

Sochiye ek **restaurant** hai. **Docker Host** = pura restaurant building. **Docker Server (Daemon)** = kitchen ka main chef jo actually khana banata hai (background mein kaam karta hai). **Docker CLI** = waiter jo aapke order (command) ko le kar chef tak pahunchata hai. Aap (user) waiter ko order dete ho (`docker run hello-world`), waiter chef ko bolta hai, chef khana banata hai (container create karta hai), aur aapko mil jata hai.

📄 **Check Docker installation:**

```
docker --version
```

Docker ka version dikhata hai — client aur server dono ka info.

Topic 8

8. Pehla Docker App — hello-world

Docker test karne ke liye ek simple image hai jiska naam hai **hello-world**. Is image ko run karne par ek simple message print hota hai — "Hello from Docker!". Ye confirm karta hai ki Docker properly install hai.

📄 **First Docker command:**

```
docker run hello-world
```

Pehli baar jab ye command chalate ho, Docker pehle locally image dhundhta hai. Local mein nahi milti toh internet se (Docker Hub se) image ko "pull" karta hai, phir us image se ek container banata hai aur hello-world message print karta hai.

Jab aap dusri baar `docker run hello-world` chalate ho, toh "pull" wala step nahi aata — kyunki image pehle hi local mein download ho chuki hai. Docker seedha local image se container bana deta hai.

🔗 **Process samjho (Behind the scenes):**

1. Aap `docker run <image>` likhte ho CLI mein → 2. CLI Docker Server (daemon) ko command bhejti hai → 3. Server pehle local mein image dhundhta hai → 4. Local mein nahi milti toh Docker Hub se pull karta hai → 5. Image local mein save hoti hai → 6. Image se container banta hai → 7. Container mein application chalti hai.

Topic 9

9. Docker Hub — Images ka Marketplace

Docker Hub (hub.docker.com) ek online marketplace hai jahan different companies aur developers apne Docker images publish karte hain. Jaise Play Store par apps milte hain, waise hi Docker Hub par images milti hain. Yahan se aap **Java**, **Node**, **MySQL**, **PostgreSQL**, **Kafka**, **Python**, **MongoDB**, **Nginx**, **Alpine**, **Ubuntu** — sab ki images pull kar sakte ho.

Agar aap `docker run java` likhoge toh error aayega — kyunki "java" naam ki image ab **deprecated** ho chuki hai. Aapko **openjdk** use karni chahiye. Lekin openjdk bhi ab deprecated hai — instructor suggest karte hain **Eclipse Temurin** ya **Amazon Corretto** use karo. Toh command banegi: `docker run eclipse-temurin`.

🔗 **Real-Life Example (Hindi):**

Docker Hub = Amazon / Flipkart jaisa online store hai. Jaise Amazon par aap alag-alag brands ke products order karte ho (Samsung ka phone, Sony ka TV, Philips ka trimmer), waise hi Docker Hub par aap alag-alag companies ke "images" order (pull) karte ho. Oracle ka Java image, MongoDB Inc. ka MongoDB image, MySQL ka image. Aur jaise Amazon par product ki **rating aur reviews** dekh kar best choose karte ho, waise hi Docker Hub par bhi **"Docker Official Image"** badge dekh kar reliable image choose karo.

📄 **Search and run images:**

```
docker run eclipse-temurin
```

Eclipse Temurin (OpenJDK) image run karta hai — Java environment banata hai.

10. Docker Tags Explained (Version system)

Har Docker image ke saath ek **tag** hota hai jo us image ka version batata hai. Tag ka format hota hai: `image_name:tag`. Agar aap `java:21` likhte ho, toh **java** image naam hai aur **21** tag (version) hai. Tag ke bina Docker by default **latest** tag use karta hai.

Examples: `openjdk:17`, `eclipse-temurin:21`, `mysql:8`, `node:20`, `ubuntu:22.04`

✂ Real-Life Example (Hindi):

Sochiye aap **Amul Butter** kharidne gaye. Dukan par aapne bola "Amul Butter do" — lekin dukan wala poochega **"kaunsa pack? 100 gram wala ya 500 gram wala?"** Yahan **"Amul Butter"** = **Image name**, aur **"100g / 500g"** = **Tag**. Agar aapne size nahi bataya toh dukan wala default ek pack de dega (jaise "latest" tag). Bilkul waise hi Docker — agar tag nahi batate ho toh by default **latest** use karta hai.

📖 Tag ke saath image run karna:

```
docker run eclipse-temurin:21
```

Specific version (Java 21) wala image chalata hai.

11. "latest" Tag ka asli matlab

Bahut log sochte hain ki agar `java:latest` likha toh humein Java ka **sabse naya version** (jaise Java 27) mil jayega. **Ye GALAT hai.** "latest" ka matlab ye nahi ki sabse latest version milega.

"latest" ka asli matlab ye hai jo **image creator (developer)** ne "latest" tag assign kiya hai. Woh developer decide karta hai ki "latest" kya hoga. Ho sakta hai "latest" ek purana version ho jo developer ne default set kiya ho. **"latest" guaranteed latest version nahi deta.**

✂ Real-Life Example (Hindi):

Sochiye ek **kirana dukan** par "latest" wala dabba rakha hai. Dukan wala ne jo dabba "latest" label lagaya hai, wohi "latest" hai — chahe woh naya ho ya purana. Ho sakta dukan wala ne ek purana stock ko hi "latest" label lagaya ho kyunki usko woh bechna hai. **Aapko guarantee nahi ki "latest" label wala dabba sach mein sabse fresh ho.** Bilkul waise hi Docker mein — image creator decide karta hai "latest" kya hoga, guaranteed newest version nahi hota.

⚠ Best Practice:

Production mein kabhi bhi **"latest" tag use mat karo.** Hamesha **specific version** likho (jaise `eclipse-temurin:21`), taaki pata rahe ki kaunsa version chal raha hai. "latest" use karne se future mein kuch toot gaya toh debug karna mushkil ho jata hai.

12. docker ps — Running containers dekhna

`docker ps` command se aap dekh sakte ho ki is waqt kaun-kaun se containers **running** hain. Isme container ID, image naam, status, aur naam dikhte hain.

📖 Container commands:

```
docker ps
```

Sirf RUNNING containers dikhata hai.

```
docker ps -a
```

Saare containers dikhata hai — running + stopped (jinhone chal ke stop ho gaye).

```
docker images
```

Aapke local mein download ki gayi saari images dikhata hai.

✂ Real-Life Example (Hindi):

`docker ps` = Hotel ke reception par poochna **"abhi kaunse rooms occupied (running) hain?"** — sirf wahi guests dikhte hain jo abhi room mein hain. `docker ps -a` = **"Aaj kaunse guests aaye the (chahe abhi checkout kar gaye ho)?"** — saare guests ki

list dikhti hai, chahe woh abhi room mein hon ya chal gaye hon.

Topic 13

13. Containers apne aap kyun stop hote hain?

Jab aap `docker run hello-world` chalate ho, toh container hello-world message print karta hai aur phir **turant stop ho jata hai**. Aisa kyun? Kyunki hello-world image ka **main process (command) khatam ho gaya**. Container tab tak chalta hai jab tak uska main process chal raha hai. Jaise hi main process finish hota hai, container bhi stop ho jata hai.

Java image (`eclipse-temurin`) ko jab run karte ho, toh woh bhi turant stop ho jata hai — kyunki uska default command ek shell start karta hai jo background mein kuch nahi karta.

✂ Real-Life Example (Hindi):

Sochiye aapne ek **worker (mistri)** ko bulaya ghar par. Mistri ne kaam kiya (jaise pankha repair kiya) aur jab kaam khatam hua, toh woh **apne aap chala gaya** (stop ho gaya). Mistri tab tak rukega jab tak uska kaam chal raha hai. **Docker container bhi waisa hi hai** — jab tak uska main "task" (process) chal raha hai, container running rahega. Task khatam = container stop.

🔧 Container running rakhne ke liye:

Aapko ek aisa process chahiye jo continuously chalta rahe — jaise ek server, ek shell, ya ek long-running application. Tabhi container running rahega.

Topic 14

14. Interactive Mode (-it flag) — Container ke andar jaana

Agar aap chahte ho ki container **running rahe** aur aap uske **andar interact** kar sako (commands chala sako, type kar sako), toh `-it` flag use karo. `-i` ka matlab **interactive** (input lena) aur `-t` ka matlab **tty** (terminal assign karna). Dono mila ke `-it` .

📄 Interactive mode:

```
docker run -it ubuntu
```

Ubuntu container chalata hai aur aap turant uske andar terminal mein aa jate ho – jaise ek naye Linux machine par login kar liya ho. Ab aap `ls`, `pwd`, `cat` jaisi commands chala sakte ho.

```
exit
```

Container se bahar aane ke liye – container stop ho jayega.

✂ Real-Life Example (Hindi):

Sochiye ek **"experience zone"** (jaise Apple store mein iPhone try karne ka zone) hai. Normal mode mein aap bahar se phone dekh sakte ho, lekin chala nahi sakte. **-it mode** mein aap andar jaate ho, phone haath mein lete ho, aur khud try kar sakte ho — apps khol sakte ho, settings change kar sakte ho. **Container ke andar -it mode mein jaana = us Linux machine ko khud chalana**.

🔑 -it flags:

- `-i` = interactive (STDIN open rakhta hai — input le sakta hai).
- `-t` = tty (terminal assign karta hai — prompt dikhta hai).
- `-it` = dono ek saath — proper interactive shell.
- Bina `-it` ke container seedha print karke stop ho jata hai.

Topic 15

15. OS, Kernel & User Space — Basics

Docker ko samajhne se pehle operating system ke 3 main parts samajhne hain:

- **Hardware:** Aapki physical machine (CPU, RAM, disk).
- **Kernel:** OS ka core jo hardware se directly baat karta hai. Ye memory manage karta hai, processes chalata hai, files handle karta

hai. Windows ka alag kernel, Linux ka alag kernel, Mac ka alag kernel.

- **User Space:** Ye woh hissa hai jahan aapke applications chalte hain — browser, IDE, games, terminal. User space kernel ke upar hota hai.

🔗 **Real-Life Example (Hindi):**

Sochiye ek **badi company** hai. **Hardware** = company ka physical office building (walls, furniture, electricity). **Kernel** = company ka **main manager / administration department** jo sab kuch control karta hai — kaunsa employee kaunse floor par baithega, electricity kaise distribute hogi, security kaise manage hogi. **User Space** = alag-alag departments (HR, Finance, IT) jo apna kaam karte hain, lekin sab manager (kernel) ke rules ke andar. **Docker user space mein kaam karta hai** — alag-alag "departments" (containers) banata hai jo kernel ko share karte hain.

🔑 **Important:**

Docker containers **apna alag kernel nahi banate** — woh host machine ke kernel ko share karte hain. Isi liye Docker lightweight hai (Virtual Machine se).

Topic 16

16. Virtual Machines vs Docker (Hypervisor)

Docker ke pehle Virtual Machines (VMs) use hote the. VM kya karta hai — ek physical machine ke upar ek **Hypervisor** software chalta hai, jo us machine ko alag-alag **virtual machines** mein baant deta hai. Har VM mein **pura OS (apna kernel + apna user space)** install hota hai. Matlab har VM mein Windows ya Linux ka GBs ka OS install karna padta tha.

Docker mein ye problem nahi hai. Docker mein har container **apna alag kernel nahi rakhta** — sab containers host machine ka kernel **share** karte hain. Sirf user space alag hota hai. Isi liye Docker container **seconds mein start** ho jata hai aur **MBs ka size** hota hai, jabki VM **minutes** mein start hota hai aur **GBs** ka size hota hai.

Virtual Machine (VM)	Docker Container
Har VM mein pura OS (apna kernel) hota hai.	Sab containers host ka kernel share karte hain.
Hypervisor chahiye.	Hypervisor nahi chahiye — Docker engine kaafi hai.
Size: GBs mein.	Size: MBs mein.
Start time: Minutes .	Start time: Seconds .
Heavy / slow.	Lightweight / fast.

🔗 **Real-Life Example (Hindi):**

Sochiye aapko ek building mein 5 alag **offices** chahiye. **VM approach** = building mein 5 alag-alag **puure ghar (independent houses)** bana lo — har ghar ka apna foundation, apni electricity meter, apna kitchen (pura OS). Bahut jagah aur paisa waste. **Docker approach** = building mein 5 alag **rooms (cabins)** bana lo, sab ek hi building (kernel) ka electricity aur paani share karte hain, har room ka apna furniture (user space) hai. Bahut kam jagah aur paisa — aur jaldi setup.

Topic 17

17. Docker Shared Kernel kaam kaise karta hai

Docker ka secret hai **shared kernel**. Sab containers same kernel use karte hain, lekin Docker **isolation** provide karta hai using Linux features jaise **namespaces** (har container ka apna process tree, network, file system) aur **cgroups** (resource limiting).

Iska matlab — ek hi Linux kernel par aap 10 alag containers chala sakte ho, har container ko lagega ki woh akela hai, lekin actually sab same kernel share kar rahe hain. **Isi liye Docker itna fast aur lightweight hai.**

⚠ **Important Limitation:**

Kyunki containers host ka kernel share karte hain, ek **Linux container Linux host par hi properly chalega**. Windows par Linux container chalane ke liye Docker internally ek **lightweight Linux VM** use karta hai. Mac par bhi same.

🔗 **Real-Life Example (Hindi):**

Sochiye ek **bada PG (paying guest) house** hai jisme 5 ladke rehte hain. Sab **ek hi kitchen (kernel) share** karte hain, ek hi gas, ek hi fridge. Lekin har ladke ka apna **alag room (container)** hai, apna alag cupboard, apni alag books. Har ladke ko lagta hai uska apna private space hai, lekin actually woh kitchen (kernel) ko share kar raha hai. **Docker bilkul waisa hi — har container ko lagta hai woh isolated hai, lekin actually kernel share ho raha hai.**

Topic 18

18. Ubuntu Container chalana aur explore karna

Ubuntu image ko interactive mode mein chala kar aap ek **pura Linux machine** ke andar aa sakte ho. Wahan `ls`, `cat /etc/os-release`, `apt-get` jaisi commands chala sakte ho. Aap container ke andar `curl` bhi install kar sakte ho (`apt-get update && apt-get install curl`).

☞ Ubuntu container explore:

```
docker run -it ubuntu
```

Ubuntu container chalu + andar terminal mein jaate ho.

```
cat /etc/os-release
```

Container ke andar Ubuntu version dekho.

```
apt-get update && apt-get install -y curl
```

Container ke andar curl install karo.

✂ Real-Life Example (Hindi):

Sochiye aapko ek **"trial room"** diya gaya jisme aap ja sakte ho, kapde try kar sakte ho, mirror dekh sakte ho. Lekin jab aap bahar aate ho, room **waisa hi rehta hai** (ya reset ho jata hai). Ubuntu container mein jaana = ek trial room mein jaana. Aap `apt-get` se kuch install kar lo, lekin container **stop aur remove** hone ke baad wapas run karoge toh sab kuch **reset** ho jayega — fresh Ubuntu milega. Changes persist karane ke liye image ko **commit** ya **Dockerfile** se build karna padta hai.

Topic 19

19. Spring Boot App ko Dockerize karna — Dockerfile

Ab hum apni Employee Service Spring Boot application ko Docker mein daalenge. Iske liye hume ek **Dockerfile** likhna hoga. Dockerfile ek text file hai jisme instructions hoti hain ki image kaise banani hai. Ye file project root (pom.xml ke saath) mein hoti hai.

☞ Sample Dockerfile for Spring Boot:

```
FROM eclipse-temurin:21
```

Base image — Java 21 environment le lo.

```
WORKDIR /app
```

Container ke andar /app directory set karo.

```
COPY target/employee-service-0.0.1-SNAPSHOT.jar app.jar
```

Aapki JAR ko container mein copy karo — naam app.jar rakh do.

```
EXPOSE 8080
```

Batado ki application port 8080 use karegi.

```
ENTRYPOINT ["java","-jar","app.jar"]
```

Container start hote hi ye command chalegi — JAR run hogi.

Dockerfile Instructions ke matlab:

- **FROM:** Base image batata hai (jaise eclipse-temurin:21).
- **WORKDIR:** Container ke andar working directory set karta hai.
- **COPY:** Aapke local file/folder ko container mein copy karta hai.
- **EXPOSE:** Documentation ke liye — batata hai kaunsa port use hoga.
- **ENTRYPOINT:** Container start hone par jo command chalegi.

✂ Real-Life Example (Hindi):

Sochiye aap ek **"recipe book" (Dockerfile)** likh rahe ho kisi chef ke liye jo aapke dish ko doosre kitchen mein bana sake. Recipe mein likhte ho: **"STEP 1:** Pehle ek kitchen le lo jisme gas stove aur basic utensils hon (FROM). **STEP 2:** Apne dabbe se kaju aur doodh us kitchen mein rakho (COPY). **STEP 3:** Gas chalu karo aur kheer banana shuru karo (ENTRYPOINT)." Chef (Docker) ye

recipe follow karke **image** (ready dish) bana lega. Phir us dish ko chaaron baar **serve** (container) kiya ja sakta hai.

Topic 20

20. Docker Image Build karna

Dockerfile likhne ke baad, usse actual image banani hai — isi ko **build** kehte hain. Build command Dockerfile padhti hai aur uske according ek image banati hai.

📦 Build Docker image:

```
docker build -t employee-service:1.0 .
```

-t se image ka naam + tag dete ho (employee-service, tag 1.0). Last mein . (dot) batata hai ki Dockerfile current directory mein hai.

Build hone ke baad **docker images** chala kar check karo — aapki image list mein dikhegi. Ab is image se container bana sakte ho.

📦 Image se container run karna:

```
docker run employee-service:1.0
```

Aapki Spring Boot application container ke andar chalu ho jayegi.

🔗 Real-Life Example (Hindi):

Sochiye aapne ek **recipe (Dockerfile)** likhi. Ab woh recipe ko **"test kitchen"** mein bheja (docker build) — chef ne recipe follow karke **ready dish (image)** banayi. Ab woh dish **"serve"** kar sakte ho chaaron logon ko (docker run — har baar ek naya container). Ek hi dish se chaaron servings mil sakti hain.

Topic 21

21. Port Mapping Explained (-p flag)

Spring Boot application container ke andar **port 8080** par chal rahi hai. Lekin aapka browser aapke host machine (laptop) se baat karta hai, container ke andar nahi. Toh agar aap browser mein **localhost:8080/employee** hit karna chahte ho, toh aapko **host ka port container ke port se map** karna hoga — isi ke liye **-p** flag use hota hai.

📦 Port mapping:

```
docker run -p 8080:8080 employee-service:1.0
```

Pehla 8080 = HOST (aapke laptop) ka port. Doosra 8080 = CONTAINER ka port. Ab browser mein localhost:8080/employee hit karoge toh request container ke 8080 par pahunchegi.

```
docker run -p 9090:8080 employee-service:1.0
```

Host ka port 9090, container ka 8080. Ab localhost:9090/employee hit karoge. Useful jab 8080 host par already busy ho.

🔗 Real-Life Example (Hindi):

Sochiye ek **big apartment building** (aapka laptop) hai jisme **100 flats (containers)** hain. Har flat ka apna internal **"intercom number"** hai (container port 8080). Lekin bahar se aane wala call **building ka main gate (host port)** pe aata hai. Receptionist (Docker) decide karta hai ki gate ka kaunsa number kis flat ke intercom se jodna hai. **Port mapping = receptionist ka kaam** — "gate ka 8080 = flat A ka 8080", "gate ka 9090 = flat B ka 8080". Bahar wala (browser) gate ka number hit karta hai, receptionist andar wale flat tak pahuncha deti hai.

⚠ Yaad rakho:

Bina **-p** ke container chalega, lekin aap browser se access nahi kar paoge — kyunki host aur container ke ports connected nahi honge.

Topic 22

22. Docker Hub par Login aur Image Push karna

Ab tak aapne local image banayi. Lekin agar aap chahte ho ki aapki image **kisi dusre laptop par** bhi chal sake, toh use **Docker Hub** par upload (push) karna padega. Pehle Docker Hub par account banao (hub.docker.com), phir terminal se login karo.

📌 Docker Hub Login & push:

```
docker login
```

Docker Hub ke username aur password se login karo.

```
docker tag employee-service:1.0 yourusername/employee-service:1.0
```

Image ko proper naam do (Docker Hub username ke saath) – warna push nahi hoga.

```
docker push yourusername/employee-service:1.0
```

Image ko Docker Hub par upload karo.

Image Naming Convention:

Docker Hub par push karne ke liye image ka naam is format mein hona chahiye: `username/repo_name:tag`. Jaise: `ayush/employee-service:1.0`. Bina username ke push karne par error aayega — kyunki Docker try karega `library/` (official namespace) mein push karne ka jisme aapka access nahi hai.

✂ Real-Life Example (Hindi):

Sochiye aapne ek **YouTube video banayi** (local image). Ab aap chahte ho ki dusre log bhi dekh sakein, toh video ko **YouTube par upload (push)** karna padega. Pehle aapko YouTube par **login** karna padega, phir video ko apne **channel ke naam ke saath** upload karna padega (jaise "ayush-tech/employee-service"). Phir koi bhi dusre desh se apne laptop par **download (pull)** karke chala sakta hai. **Docker Hub = YouTube for Docker Images.**

Topic 23

23. Image Naming Convention (Detail)

Pura image name format: `[registry/]username/repo[:tag]`

- **registry:** Default Docker Hub (docker.io). Aur bhi registries ho sakti hain (AWS ECR, Google GCR, private registry).
- **username:** Aapka Docker Hub username (ya "library" agar official image hai).
- **repo (repository name):** Image ka naam (jaise employee-service).
- **tag:** Version (jaise 1.0, latest). Default "latest".

✂ Real-Life Example (Hindi):

Pura address sochiye — **"Bharat (registry) / UP (state/username) / Lucknow (city/repo) / Gomti Nagar (tag)"**. Bilkul waise hi **"docker.io / yourusername / employee-service : 1.0"**. Har hissa alag cheez batata hai. Push karne se pehle saare parts sahi honi chahiye.

Topic 24

24. Detached Mode (-d) — Background mein chalana

Ab tak jo container chalaye, woh foreground mein chal rahe the — terminal block ho jata tha. Agar aap chahte ho ki container **background mein chale** aur terminal free rahe, toh `-d` flag use karo (detached mode).

📌 Detached mode:

```
docker run -d -p 8080:8080 employee-service:1.0
```

Container background mein chalega, terminal free rahega. Aapko sirf container ID print hoga.

```
docker ps
```

Check karo container running hai ya nahi.

```
docker logs <container_id>
```

Background container ke logs dekho.

✂ Real-Life Example (Hindi):

Sochiye aap **kitchen mein khaana pakaa rahe ho**. **Foreground mode** = aap gas ke saamne khade ho, jab tak khaana nahi ban jata tab tak wahi raho (terminal block). **Detached mode (-d)** = aap gas on karke **baith ke TV dekhte ho**, khaana background mein pakta rahega, aap time pe aa ke check kar lo. Bilkul waise hi `-d` se container background mein chalta hai, aap terminal pe aur kaam kar sakte ho.

25. Docker + Microservices = Perfect Match

Microservices architecture mein ek badi application ko chhote-chhote services mein tod diya jata hai — ek service user management karti hai, dusri payment karti hai, teesri notification bhejti hai. Har service apni alag technology use kar sakti hai — ek Java mein, dusri Python mein, teesri Node.js mein.

Docker microservices ke liye perfect hai kyunki har microservice ko apne alag container mein daal sakte ho — har container apna environment (Java/Python/Node + dependencies) rakhta hai, sab isolated hain, sab apne port par chalte hain. Deploy karna, scale karna, update karna — sab easy ho jata hai.

✂ Real-Life Example (Hindi):

Sochiye ek **bada hospital** hai. Pehle ek hi "**doctor**" sab kuch karta tha — OPD, surgery, X-ray, medicine — sab. (Monolith). Ab hospital ko **alag-alag departments** mein tod diya — OPD department (general patients), Surgery department (operations), Radiology department (X-ray/MRI), Pharmacy (medicines). Har department ka apna staff, apna equipment, apna room — lekin sab ek hospital building mein. **Har department = ek Docker container**, pura hospital = aapka system. Naya doctor add karna easy (scale), ek department band karna easy (stop container), sab independent.

🔑 Docker + Microservices benefits:

- Har microservice apne alag container mein — **independent deploy & scale**.
- Alag-alag technologies (Java, Python, Node) ek saath chal sakti hain.
- Koi ek service down ho toh baaki **unaffected**.
- Version update karna easy — sirf us container ko replace karo.
- CI/CD pipeline mein Docker images bahut useful.

26. Docker Images are Immutable (nahi badalti)

Docker images **immutable** hoti hain — matlab ek baar image ban gayi toh woh **nahi badalti**. Agar aap image ko update karna chahte ho, toh aapko **nayi image build** karni padegi (naya version / naya tag).

Example: Agar aapne `employee-service:1.0` build ki, aur ab code mein changes kiye, toh purani image waise hi rahegi. Aapko `employee-service:1.1` (naya tag) build karna hoga. **Purani image change nahi hogi.**

✂ Real-Life Example (Hindi):

Sochiye aapne ek "**printed book**" (**image**) publish ki — jaise "Recipe Book Edition 1". Ab book print ho gayi, **print hui book ko aap badal nahi sakte** (immutable). Agar aap recipe mein changes karna chahte ho, toh **naya edition (Edition 2) print karna padega** (naya image build). Purani edition waise hi rahegi — koi jo Edition 1 rakhta hai, uske paas wahi purani recipe hogi. **Docker image = printed edition. Update = naya edition build karna.**

🔑 Why immutable is good:

Immutability se **consistency** milti hai — ek image jo aapke laptop par hai, woh exactly same production par bhi chalegi. Koi surprise nahi. Yahi Docker ka biggest advantage hai — "build once, run anywhere".

27. Essential Docker Commands (stop, rm, prune)

Ab tak humne `docker run`, `docker build`, `docker ps` seekhe. Ab kuch aur zaroori commands:

📌 Essential Docker Commands:

```
docker stop <container_id>
```

Running container ko gracefully stop karta hai.

```
docker start <container_id>
```

Stopped container ko wapas start karta hai.

```
docker restart <container_id>
```

Container ko restart karta hai.

```
docker rm <container_id>
```

Stopped container ko permanently delete karta hai.

```
docker rm -f <container_id>
```

Running container ko force-delete karta hai.

```
docker rmi <image_id>
```

Local image ko delete karta hai.

```
docker system prune
```

Saare unused/stop containers, dangling images, networks clean karta hai – disk space bachata hai.

```
docker logs <container_id>
```

Container ke logs dekho.

```
docker exec -it <container_id> bash
```

Pehle se running container ke andar jao (exec).

```
docker pull <image>
```

Image ko Docker Hub se download karo (run kiye bina).

```
docker images
```

Local saari images list karo.

```
docker ps -a
```

Saare containers (running + stopped) list karo.

✂ Real-Life Example (Hindi):

`docker stop` = TV ka **remote se power off** karna (TV band, lekin wapas on ho sakta hai). `docker rm` = TV ko ghar se **hamesha ke liye nikal dena** (delete). `docker system prune` = ghar ki "**safai**" (**spring cleaning**) — sab kachra, purane boxes, unused furniture bahar phenk do, space ban jayega.

Topic 28

28. Quick Revision Summary (Ek nazari mein)

🔗 Docker Ek Line Mein

Docker ek tool hai jo aapki application + uska pura environment (OS, dependencies, runtime) ek saath package karke ek **Image** banata hai, aur us Image ko run karke ek isolated **Container** banata hai jisme application "**build once, run anywhere**" chale.

Key Concepts ek jagah:

- **Problem:** JAR dusre laptop par nahi chalti (environment mismatch).
- **Solution:** Pura environment package karke do — Docker.
- **Image:** Blueprint / package (JAR + JDK + OS + deps).
- **Container:** Image run karke bana running, isolated environment.
- **Dockerfile:** Recipe / instructions to build image.
- **Docker Hub:** Online marketplace for images.
- **Tag:** Image ka version (image:tag).
- **-it:** Interactive terminal.
- **-d:** Detached / background mode.
- **-p:** Port mapping (host:container).
- **Shared Kernel:** Containers host ka kernel share karte hain — lightweight.
- **Immutable:** Image ek baar bani toh nahi badalti — naya version naya tag.

Most Used Commands ek jagah:

```
docker run hello-world # test docker
```

```
docker run -it ubuntu # interactive ubuntu
```

```
docker run -d -p 8080:8080 myapp:1.0 # detached + port mapped
```

```
docker ps # running containers
```

```
docker ps -a # all containers
```

```
docker images # all local images
docker build -t myapp:1.0 . # build image from Dockerfile
docker stop <id> # stop container
docker rm <id> # remove container
docker rmi <image> # remove image
docker logs <id> # see logs
docker exec -it <id> bash # enter running container
docker pull <image> # download image
docker push <username>/<image>:<tag> # upload to Docker Hub
docker login # login to Docker Hub
docker system prune # cleanup everything unused
```

Interview ke liye quick questions:

- **Q: Docker kya hai?** Aapko environment mismatch problem solve karne wala tool — application + environment ko image mein package karta hai.
- **Q: Image vs Container?** Image = blueprint (static), Container = running instance (dynamic).
- **Q: VM vs Docker?** VM mein har machine ka apna OS (heavy), Docker mein sab host ka kernel share (lightweight).
- **Q: "latest" tag ka matlab?** Image creator ne jo "latest" set kiya — guaranteed newest version nahi.
- **Q: Dockerfile kya hai?** Text file jisme image banane ki instructions (FROM, COPY, ENTRYPOINT etc.).
- **Q: -d flag?** Detached mode — container background mein chalta hai.
- **Q: -p 8080:8080?** Host ka 8080 port container ke 8080 port se map karta hai.

💡 Final Thought (Video se):

"Ab se JAR files share mat karo — Docker Images share karo."

Jab tak aap sirf JAR doge, environment problem rahega. Docker image do toh sab kuch saath mein aayega — application guaranteed chalegi.

🔗 Final Real-Life Example (Hindi):

Pehle: Aap dost ko sirf **"recipe likh kar WhatsApp par bhejte the"** — dost ke paas ingredients nahi the toh dish nahi banti. Ab: Aap **"pura packed tiffin (Docker Image)"** bhejte ho — dost ko sirf garam karna hai (container run karna hai), dish turant ready! **Yahi Docker ka magic hai.** 🍷

Notes prepared from YouTube video: "Learn Docker in 2 Hours | Java & Spring Boot Dev → Docker PRO" by SeleniumExpress

Video URL: <https://www.youtube.com/watch?v=mBkA2OzAS68>

These notes are for study purpose — har topic ke saath Hindi real-life examples diye gaye hain.